UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Koneksi ke Database MySQL Menggunakan PDO

Donny Reza, M.Kom

Mata Kuliah Aplikasi Teknologi Online

Program Studi Teknik Informatika, Fakultas Teknik dan Ilmu Komputer
Universitas Komputer Indonesia

Definisi PDO

PDO (*PHP Data Objects*) adalah *library* bawaan PHP yang menyediakan antarmuka (*interface*) yang seragam untuk mengakses berbagai jenis database, termasuk MySQL, PostgreSQL, SQLite, dan lainnya.

Dengan PDO, kode koneksi dan *query* yang ditulis dapat digunakan pada berbagai jenis database hanya dengan mengubah konfigurasi koneksinya saja.

Dibandingkan dengan cara lama menggunakan **`mysql_*`** atau **`mysqli_*`**, PDO memiliki beberapa keunggulan:

- Mendukung banyak jenis database
- Mendukung *prepared statements* untuk mencegah *SQL Injection*
- Penanganan *error* yang lebih baik menggunakan *exceptions*
- Kode lebih bersih dan mudah dipelihara

Persiapan Database

Sebelum membuat *script* PHP, siapkan terlebih dahulu database dan tabel yang akan digunakan. Jalankan script SQL berikut di phpMyAdmin atau MySQL client yang digunakan.

```
CREATE DATABASE IF NOT EXISTS db_kampus;

USE db_kampus;

CREATE TABLE mahasiswa (
    id        INT PRIMARY KEY AUTO_INCREMENT,
    nim       VARCHAR(10)  NOT NULL UNIQUE,
    nama      VARCHAR(100) NOT NULL,
    email     VARCHAR(100) NOT NULL,
    jurusan   VARCHAR(100) NOT NULL,
    angkatan YEAR          NOT NULL
);
```

Membuat Koneksi ke MySQL

Koneksi ke database dibuat dengan membuat objek baru dari *class* PDO. *Constructor* PDO menerima tiga parameter utama, yaitu DSN (*Data Source Name*), *username*, dan *password*.

```php
<?php
$host     = "localhost";
$nama_db  = "nama_database";
$username = "root";
$password = "isipassword";

$dsn = "mysql:host=". $host.";dbname=".$nama_db.";charset=utf8";

$pdo = new PDO($dsn, $username, $password);
```

DSN adalah string yang berisi informasi tentang jenis database dan parameter koneksinya.
Untuk MySQL, DSN terdiri dari:

- mysql: -> untuk menentukan jenis database
- host= -> alamat server database
- dbname= -> nama database yang digunakan
- charset= -> encoding karakter yang digunakan

Menangani Error Koneksi

Jika koneksi gagal, PDO akan melempar *exception* bertipe PDOException. Oleh karena itu, proses koneksi sebaiknya selalu dibungkus dengan blok *try-catch*.

```php
<?php
$host     = "localhost";
$nama_db  = "nama_database";
$username = "root";
$password = "isipassword";

$dsn = "mysql:host=". $host.";dbname=".$nama_db.";charset=utf8";

try {
    $pdo = new PDO($dsn, $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Koneksi berhasil.";
} catch (PDOException $e) {
    echo "Koneksi gagal: ".$e->getMessage();
}
```

Perintah setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION) digunakan untuk mengaktifkan mode *error* berupa *exception*, sehingga setiap error yang terjadi pada operasi database akan dilempar sebagai PDOException dan dapat ditangkap menggunakan *catch*.

Output Jika Koneksi Berhasil

```
Koneksi berhasil.
```

Output Jika Koneksi Gagal

```
Koneksi gagal: SQLSTATE[HY000] [1049] Unknown database 'nama_database'
```

File Konfigurasi Koneksi

Agar koneksi database dapat digunakan di banyak *file* tanpa perlu menulis ulang, sebaiknya konfigurasi koneksi disimpan dalam satu file terpisah dan dipanggil menggunakan require atau include.

proses_upload.php

```php
<?php
$host     = "localhost";
$nama_db  = "nama_database";
$username = "root";
$password = "isipassword";

$dsn = "mysql:host=". $host.";dbname=".$nama_db.";charset=utf8";

try {
    $pdo = new PDO($dsn, $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Koneksi berhasil.";
} catch (PDOException $e) {
    die("Koneksi gagal: ".$e->getMessage());
}
```

Fungsi die() pada file koneksi digunakan untuk menghentikan eksekusi *script* sekaligus menampilkan pesan error apabila koneksi gagal, sehingga *script* tidak dilanjutkan dalam kondisi tanpa koneksi database.

proses_upload.php

```php
<?php

require "koneksi.php";

echo "Koneksi siap digunakan";
```

Menjalankan Query

PDO menyediakan dua cara untuk menjalankan *query*, yaitu menggunakan method query() untuk *query* sederhana tanpa input dari pengguna, dan menggunakan *prepared statements* untuk *query* yang melibatkan input dari pengguna.

Contoh menggunakan query()

```php
<?php

require "koneksi.php";

$result = $pdo->query("SELECT * FROM mahasiswa");

while($data = $result->fetch(PDO::FETCH_ASSOC)) {
    echo "NIM    : ".$data["nim"]."<br>";
    echo "Nama    : ".$data["nama"]."<br>";
    echo "---<br>";
}
```

Output (Contoh):

```
NIM  : 10123001
Nama : Budi Santoso
---
NIM  : 10123002
Nama : Sari Dewi
---
```

Metode fetch(PDO::FETCH_ASSOC) digunakan untuk mengambil satu baris data dari hasil query dalam bentuk array asosiatif, di mana *key*-nya adalah nama kolom. Pemanggilan dilakukan berulang di dalam *while* hingga seluruh baris habis terbaca.

Menggunakan fetchAll()

Jika ingin mengambil seluruh data sekaligus ke dalam sebuah array, gunakan fetchAll().

Contoh menggunakan fetchAll()

```php
<?php
require "koneksi.php";

$result     = $pdo->query("SELECT * FROM mahasiswa");
$mahasiswa  = $result->fetchAll(PDO::FETCH_ASSOC);

foreach ($mahasiswa as $mhs) {
    echo "NIM  : ".$mhs["nim"]."<br>";
    echo "Nama : ".$mhs["nama"]."<br>";
    echo "---<br>";
}
```

Output (Contoh):

```
NIM  : 10123001
Nama : Budi Santoso
---
NIM  : 10123002
Nama : Sari Dewi
---
```

Perbedaan fetch() dan fetchAll():
fetch() mengambil satu baris per pemanggilan sehingga cocok untuk data besar karena lebih hemat memori, sedangkan fetchAll() mengambil seluruh data sekaligus ke dalam array sehingga lebih praktis untuk data yang tidak terlalu banyak.

Menggunakan *Prepared Statement*

Prepared statements adalah cara yang disarankan untuk menjalankan *query* yang melibatkan input dari pengguna. Dengan *prepared statements*, nilai input tidak langsung disisipkan ke dalam string *query*, melainkan dikirim secara terpisah, sehingga mencegah serangan SQL Injection.

Ada dua cara untuk menyisipkan nilai pada prepared statements, yaitu menggunakan *named parameters* dan *positional parameters*.

Named Parameters

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Named parameters menggunakan nama dengan awalan titik dua (:) sebagai penanda posisi nilai di dalam *query*.

```php
<?php
require "koneksi.php";

$nim = "10123001";

$stmt = $pdo->prepare("SELECT * FROM mahasiswa WHERE nim = :nim");

// eksekusi query
$stmt->execute([":nim" => $nim]);

//ambil data menggunakan fetch()
$mhs = $stmt->fetch(PDO::FETCH_ASSOC);

if ($mhs) {
    echo "NIM   : ".$mhs["nim"]."<br>";
    echo "Nama  : ".$mhs["nama"]."<br>";
    echo "Email : ".$mhs["email"];
} else {
    echo "Data tidak ditemukan.";
}
```

Output (Contoh):

```
NIM   : 10123001
Nama  : Budi Santoso
Email : budi@email.com
```

Positional Parameters

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Positional parameters menggunakan tanda tanya (?) sebagai penanda posisi nilai di dalam *query*. Nilai dikirimkan dalam bentuk array berurutan sesuai posisi tanda (?)

```php
<?php
require "koneksi.php";

$nim   = "10123001";
$nama  = "Budi Santoso";

$stmt = $pdo->prepare("SELECT * FROM mahasiswa WHERE nim = ? AND nama = ?");

$stmt->execute([$nim, $nama]);

$mhs = $stmt->fetch(PDO::FETCH_ASSOC);

if ($mhs) {
    echo "Data ditemukan: ".$mhs["nama"];
} else {
    echo "Data tidak ditemukan.";
}
```

Perhatikan jumlah (?) di sini

Jumlah variabel atau isi pada array sebagai parameter execute() harus sama dengan jumlah (?) di atas.

Pada contoh, $nim akan dipetakan pada ? pertama, dan $nama pada ? kedua

Perbedaan *named* dan *positional parameters*:
named parameters lebih mudah dibaca karena nama parameternya deskriptif, sedangkan *positional parameters* lebih ringkas namun urutan nilainya harus benar-benar sesuai dengan urutan tanda ? di dalam *query*.

Operasi CRUD

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

CRUD adalah singkatan dari Create, Read, Update dan Delete, yang merupakan 4 operasi dasar yang umum dilakukan pada database.

Pada MySQL, setiap operasi memiliki perintah SQL yang berbeda.

Create menggunakan INSERT, contoh:

```
INSERT INTO mahasiswa(“10100246”, “Yaris Riyadi”);
```

Read menggunakan SELECT, contoh:

```
SELECT * FROM mahasiswa;
```

Update menggunakan UPDATE, contoh:

```
UPDATE mahasiswa SET nama=“Robby Darwis” WHERE nim=“10100246”;
```

Delete menggunakan DELETE, contoh:

```
DELETE FROM mahasiswwa WHERE nim=“10100246”;
```

Create – Menambah Data

```php
<?php
require "koneksi.php";

$nim   = "10123004";
$nama  = "Rina Marlina";
$email = "rina@email.com";

try {
    $stmt = $pdo->prepare(
        "INSERT INTO mahasiswa (nim, nama, email)
         VALUES (:nim, :nama, :email)"
    );

    $stmt->execute([
        ":nim"   => $nim,
        ":nama"  => $nama,
        ":email" => $email
    ]);

    echo "Data berhasil ditambahkan. "
         . "ID baru: ".$pdo->lastInsertId();

} catch (PDOException $e) {
    echo "Gagal menambahkan data: ".$e->getMessage();
}
```

Output (Contoh):

```
Data berhasil ditambahkan. ID baru: 4
```

Metode lastInsertId() digunakan untuk mendapatkan nilai *auto increment* dari data yang baru saja ditambahkan.

Jika pada tabel, id terakhir adalah 10, maka setelah data berhasil ditambahkan, lastInsertId() akan mengembalikan/memberikan nilai 11 yang merupakan id baru.

Read – Membaca Data

```php
<?php
require "koneksi.php";

try {
    $stmt = $pdo->prepare(
        "SELECT * FROM mahasiswa ORDER BY nama ASC"
    );
    $stmt->execute();

    $mahasiswa = $stmt->fetchAll(PDO::FETCH_ASSOC);

    foreach ($mahasiswa as $mhs) {
        echo "NIM   : ".$mhs["nim"]."<br>";
        echo "Nama  : ".$mhs["nama"]."<br>";
        echo "Email : ".$mhs["email"]."<br>";
        echo "---<br>";
    }

} catch (PDOException $e) {
    echo "Gagal mengambil data: ".$e->getMessage();
}
```

Output (Contoh):

```
NIM   : 10123003
Nama  : Andi Pratama
Email : andi@email.com
---
NIM   : 10123001
Nama  : Budi Santoso
Email : budi@email.com
---
NIM   : 10123004
Nama  : Rina Marlina
Email : rina@email.com
---
```

Update – Mengubah Data

```php
<?php
require "koneksi.php";

$nim       = "10123001";
$email_baru = "budi.baru@email.com";

try {
    $stmt = $pdo->prepare(
        "UPDATE mahasiswa SET email = :email WHERE nim = :nim"
    );

    $stmt->execute([
        ":email" => $email_baru,
        ":nim"   => $nim
    ]);

    echo "Data berhasil diubah. "
       . "Jumlah Data (Baris) terpengaruh: ".$stmt->rowCount();

} catch (PDOException $e) {
    echo "Gagal mengubah data: ".$e->getMessage();
}
```

Output (Contoh):

```
Data berhasil diubah. Jumlah Data (Baris) terpengaruh: 1
```

Metode rowCount() digunakan untuk mengetahui jumlah baris yang terpengaruh oleh query UPDATE, INSERT, atau DELETE.

Delete – Menghapus Data

```php
<?php
require "koneksi.php";

$nim = "10123004";

try {
    $stmt = $pdo->prepare(
        "DELETE FROM mahasiswa WHERE nim = :nim"
    );

    $stmt->execute([":nim" => $nim]);

    echo "Data berhasil dihapus. "
       . "Jumlah Data yang dihapus: ".$stmt->rowCount();

} catch (PDOException $e) {
    echo "Gagal menghapus data: ".$e->getMessage();
}
```

Output (Contoh):

```
Data berhasil dihapus. Jumlah Data yang dihapus: 1
```

Menutup Koneksi

Koneksi PDO akan otomatis ditutup saat script selesai dieksekusi. Namun, jika ingin menutup koneksi lebih awal secara eksplisit, cukup set variabel PDO menjadi null.

```
<?php
require "koneksi.php";

// ... lakukan operasi database ...

// Menutup koneksi
$pdo = null;

echo "Koneksi telah ditutup.";
```

Ringkasan Method (Metode) PDO

UNIKOM
DIGITAL ENTREPRENEURIAL UNIVERSITY

Metode (Method)	Kegunaan
`new PDO($dsn, $user, $pass)`	Membuat koneksi ke database
`setAttribute()`	Mengatur atribut perilaku PDO
`query()`	Menjalankan query tanpa parameter
`prepare()`	Menyiapkan *prepared statement*
`execute()`	Menjalankan (eksekusi) *prepared statement*
`fetch()`	Mengambil satu baris hasil *query*
`fetchAll()`	Mengambil semua baris hasil *query*
`lastInsertId()`	Mendapatkan ID terakhir yang disisipkan
`rowCount()`	Mendapatkan jumlah baris yang terpengaruh
`$pdo = null`	Menutup koneksi database